

RESEARCH ARTICLE

WILEY

SQVDT: A scalable quantitative vulnerability detection technique for source code security assessment

Junaid Akram¹ | Ping Luo

Key Laboratory of Information System Security, School of Software, Tsinghua University, Beijing, China

Correspondence

Junaid Akram, Key Laboratory of Information System Security, School of Software, Tsinghua University, Beijing, Haidian District, China.
Email: znd15@tsinghua.org.cn

Summary

Vulnerability detection and exploit is becoming a very important part of security, especially in malware code delivery, hacking a system, efforts to create patches, improving the source code, or updating a software. Vulnerabilities in applications, including browsers, media players, online services, document readers, and so forth, are often exploited and cause a serious damage. In this article, we propose a vulnerability detection technique to detect vulnerabilities in software, as well as shared libraries at source code level. We crawl the vulnerable source code by tracing and locating the patch files from different web sources according to their CVE-numbers and built a fingerprint index of 2931 vulnerable files. Then we developed a vulnerability detection approach based on code clone detection technique and detect hundreds of vulnerabilities in thousands of **GitHub open source projects**, which are not noticed before as vulnerable. We detected vulnerabilities in some very famous recently available software, including latest version of Linux, HTC-kernel, FindX-8.1-kernel, and in 7-TB of **C/C++ source code** (152,823 open source projects). In this study, we discuss some of the very high severity level (CVSS) vulnerabilities that are detected by our approach. Furthermore, we performed an empirical evaluation and verification on these vulnerabilities, including intraproject clone vulnerabilities, copied-kernel clone vulnerabilities, and library-used clone vulnerabilities. Our technique is very fast, efficient, reliable, practical, scalable, and can be implemented at industrial level. The comparison with the state-of-the-art tools shows the effectiveness of our approach.

KEYWORDS

big code, clone detection, code vulnerability detection, software reuse, software security and maintenance

1 | INTRODUCTION

Recently, a lot of effort has been devoted in software security field to reduce the impact of software vulnerabilities. Source code security assurance is a process, which helps us to design a software that can protect the resources and the data in it. As the number of threats in software systems are increasing, the security of them must be assured, which indirectly fully depends on the quality of the source code. In the field of information technology, the software security assurance is a key element for business continuity, personal data security, national security, cybersecurity, homeland security, and internet

security. A single exploit vulnerability can actually cause many damages to an organization. Thus, it is very paramount to detect and mitigate the software vulnerability before its release.

A security risk is more often classified as vulnerability. Vulnerability exploits are an important vector for malware infection. A bug of CVE-2017-5638 was revealed in March 2017, but the hackers were using it until May 2017 against Equifax¹, where they were stealing the personal information of over 143 million users in the US, including names, birth (places and dates), social security numbers, and so forth.¹ Another bug CVE-2018-1274 was released in April 10, 2018 of very critical severity, which causes a denial of services (DOS) attack to an authenticated remote user for parsing on memory and CPU consumption. It was fixed in April 17, 2018². But many users are still unaware of this vulnerability. Some normal users, who do not have computer knowledge, just ignore the patch files. In many cases some developers used the same code fragment copied from other software systems without checking whether the code is clean from vulnerabilities or not. These kind of copied code fragments are called as cloned code, which is quite obvious and adopted approach in developers. Another vulnerability of high severity CVE-2018-1000199, was published in April 2018 in Linux Kernel version of 3.18. It modifies *user_hw_breakpoint()*, which can result in a big crash and memory corruption. This attack actually appears to be exploitable through the local code execution. This vulnerability is already fixed in git platform of commit id: f67b15037a7a50c57f72e69a6d59941ad90a0f0f³.

Vulnerability detection and creation of patch files have improved the ability of fast overcoming the security threat in software systems. An automatic exploit is possible through patch files.²⁻⁴ Patches to vulnerable code are very important to make sure security measures.⁵⁻⁹ Previous research work shows that whenever a patch is released, an attacker (Hacker) can easily use this patch to find exact location of this buggy code in a very short time and can automatically create an exploit.¹⁰ Recently, open source software (OSS) are increasing the rate of vulnerabilities because most of the software developers copy/paste the code without modifying it. It is very adapting approach during software development. However, it is quite difficult to recognize and differentiate the source code to say which is original and which is copied.¹¹

Code clones in source code always bring a lot of troubles in software security maintenance.¹² Previous research shows that almost 7%–23% of source code is cloned in large systems.¹³ Even recent work¹⁴ shows that 22.3% of Linux code has been cloned. This cloned code can be a derived from previous versions of Linux or from other open source sources. There can be a possibility that this cloned code is vulnerable, that have been repaired through a patch file in the original source file but still remain unpatched in other versions.¹⁰ In particular, OSS is a widely used code-base during development process. Thus, code cloning (copied code) actually becomes the major cause of vulnerability. For example, “OpenSSL” a heart-bleed vulnerability of CVE-2014-0160, affected many types of systems, including websites, web applications, operating systems and servers. These affected systems were using either the whole library of “OpenSSL” or a part of it.¹⁵

Motivation: The following reasons motivate us to detect vulnerabilities in different projects at source code level.

- The increase of ubiquitous use of OSS has increase the software vulnerabilities mainly caused by code cloning, which led to a big threat.
- It is quite hard for security auditors, to audit the source code for vulnerability detection at large-scale level.
- Most of the software, directly or indirectly depends on the external components including libraries, utilities, and programs, which can be faulty or vulnerable. Such a big amount of components reuse is potentially risk oriented. An analysis shows that there are vulnerabilities in almost 0.62% Maven projects.¹⁶
- However, most of the researchers have been working on model-based vulnerability detection, which focused on the prevention from software vulnerabilities by predicting their future appearance.^{17,18} There is a very less research that has been focused on a scalable quantitative vulnerability detection at source code level.
- Focusing on the detection of vulnerable source code based on its CVE-number.
- Minimizing the false detection rate by focusing on exact match detection (vulnerable code).

A single exploited software vulnerability can cause many damages to an organization. Annual losses caused by cyber-attacks have been reported as \$226 billion through world-wide. WannaCry⁴ was an attack in year 2017, which crippled many computers in at least 150 countries and cost in billions of dollars. A cyber risk modeling organization “Cyence”

¹<https://www.engadget.com/2017/09/13/equifax-apache-argentina/>

²<https://pivotal.io/security/cve-2018-1274>

³<https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2018-1000199>

⁴<https://www.cbsnews.com/news/wannacry-ransomware-attacks-wannacry-virus-losses>

estimates the cost about 4 billion dollars. The way it spreads from one system to another was really surprised cybersecurity experts. Data always remain the primary target for hackers. Microsoft has been predicted that by year 2020, the online data volumes will be 50 times greater than today. Almost 111 billion LOC being produced every year⁵, which is going to include billions of software vulnerabilities that can be exploited.

In this article, we present scalable quantitative vulnerability detection technique (SQVDT), a fast, flexible, reliable and scalable vulnerability detection technique, which detects vulnerable source code files on the basis of code clone detection approach for the source code security assessment. The vulnerable source code files are retrieved from different web source code repositories by tracing the patch files of those vulnerabilities. From patch files, the vulnerable source code files are retrieved against its CVE-numbers, which are used as a signature set to detect vulnerabilities in any subject system (testing projects). We adopt a novel mechanism for fingerprint indexing, of vulnerable source code at file level granularity. We indexed fingerprint of almost 3000 vulnerable files as well as 7-TB (C/C++) of source code (open source projects). Then we detect vulnerabilities in almost 152,823 open source projects, including three top rated projects of *HTC-kernel*, *FindX-8.1-kernel*, and latest version of *Linux-5.3.7*. Some of the vulnerabilities in these projects are mentioned in this article, as a reference with respect to CVE-numbers. The detected vulnerabilities in these open source projects are not announced before, which were using it as a library or by copying the code from other open source systems. Experiments in this article, proved the efficiency and scalability of our approach. Following are the main contributions in this article.

- Extracting and building a vulnerability source code benchmark by tracing the patch files.
- Developing a vulnerability detection approach (SQVDT) based on code clone detection technique.
- Detection of vulnerabilities in some very famous OSS.
- Performing an empirical evaluation and verification study on these detected vulnerabilities.
- Implicating of patch-based static analysis on these vulnerabilities.
- Displaying results of many detected vulnerabilities in 7-TB of source code (152,823 projects).

2 | FRAMEWORK OVERVIEW AND TERMINOLOGIES

For vulnerability detection at large-scale level, we used MapReduce rule of divide and conquer, which help us to overcome on some of the shortcoming of available approaches. These shortcomings include scalability, flexibility, speed, simplicity, and so forth. MapReduce is a paradigm that enables a massive scalability over hundreds of servers in a Hadoop cluster. As a processing component, it is the heart of Hadoop. The term MapReduce, refers to two distinct tasks which the Hadoop performs. The first task is the map, which takes data and convert it into another form of data, where every element breaks down into some value pairs. The reduce task takes the output of map as an input and combines these value pairs into some smaller set of tuples. As the name MapReduce implies that the reduce task is always performed after the map task. In the setting up framework to conduct our research, we used different software, tools, plugins, and so forth. The main parts of the framework are Hadoop, MapReduce, HDFS, and HBase. We have setup this framework as a single-node Hadoop installation and multinode Hadoop installation. We made some configuration in Hadoop, HBase, Zookeeper, HDFS NameNode, DataNode, and so forth, on every machine, including setting up the Java path, Zookeeper path, Hadoop path, HBase path, IP addresses, and so forth. During detection process, the thread controls the main node module, which calls the functions of the control layer to complete the detection process. Furthermore, the task monitoring was carried out at any time because a detection task corresponds to a thread. All of the machines involved in experiments were installed Linux operating system of Ubuntu 14.04 LTS. The system specifications were, as a single-node Master (Intel Core-i7, 3.60 GHz*8, 32 GB RAM) and as a multinode Slave (Intel Core-i5, 3.60 GHz*4, 16 GB RAM). We used IntelliJ-IDEA and Java language as an integrated development environment for developing our vulnerability detection platform. Here are some terms that are used in this article.

Vulnerability: It is a weakness or a flaw in software system, which allows an attacker (hacker) to overcome a system's security.

NVD: NVD (National Vulnerability Database) ⁶ is the United States (government) standards-based vulnerability repository. It includes security references, flaws, security misconfigurations, product names and its metrics.

⁵<https://www.csoonline.com/article/3110467/security/cybercrime-damages-expected-to-cost-the-world-6-trillion-by-2021.html>

⁶<https://nvd.nist.gov/vuln/search>

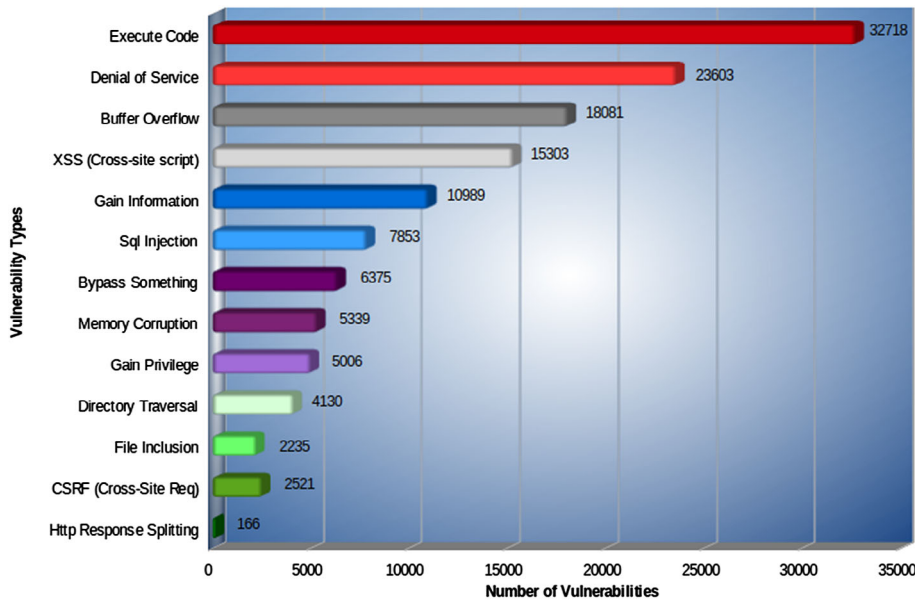


FIGURE 1 Total number of vulnerabilities by type (1999 - March 2020) [Color figure can be viewed at wileyonlinelibrary.com]

CVE: It is the common vulnerabilities exposures⁷ system, which provides a reference CVE-IDs for publicly known vulnerabilities, including a unified identification number, description, and a reference number for known vulnerabilities.

Code Clones: A copied code fragment from other systems is called a code clone. There are four different types of code clones,¹¹ that is, Type-1: Exact clones, Type-2: Renamed clones, Type-3: Restructured clones, and Type-4: Semantic clones.

Type-1: Identical code fragments but may have some variations in whitespace, layout, and comments.

Type-2: Syntactically equivalent fragments with some variations in identifiers, literals, types, whitespace, layout, and comments.

Type-3: Syntactically similar code with inserted, deleted, or updated statements.

Type-4: Semantically equivalent, but syntactically different code.

Token: A small preprocessed unit that a compiler can understand, that is, in `(int x = 10 ;)` there are 5 tokens

Chunk: A set of tokens.

Indexing: Indexing consists on fingerprint hash values of vulnerable source code.

3 | VULNERABLE CODE EXTRACTION

Vulnerable code is the code that has security flaws and open to attack to cause harm. There are different types of vulnerabilities in the source code, that is, injection vulnerabilities, buffer overflow, sensitive data exposure, broken authentication, session management security misconfiguration, and so forth. Every few years, the OWASP (Open Web Application Security Project)⁸ releases a list of top ten critical security risks. OWASP members examine the occurrence rate and the severity of the threat. The primary purpose of this is to help the developers to implement better security measures. Total number of vulnerabilities happened since the year 1999 with respect to their type are shown in Figure 1. This figure also shows the common vulnerabilities trends and the security risks of the software. The data collected from CVE details⁹, which is the ultimate security vulnerability database. From Figure 1, we can see that a large number of “execute code” vulnerabilities have been detected, in which an attacker execute commands of his choice on a target machine. It gives an attacker a chance to execute an arbitrary code. Most of our extracted vulnerabilities source code dataset consists on these vulnerabilities.

Patch File: It is a series of instructions in the source code form that describes how to change a file or a set of files in an application. A patch file CVE-2018-1000199¹⁰ is shown in Figure 2, which has been released to fix a crash and memory corruption problems in *Linux-kernel* version 3.18. In Figure 2, the “+” symbol shows the addition in the code, the “-”

⁷<https://cve.mitre.org/>

⁸https://www.owasp.org/index.php/Top_10_2013-Top_10

⁹<https://www.cvedetails.com/vulnerabilities-by-types.php>

¹⁰<https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2018-1000199>

FIGURE 2 Diffstat of CVE-2018-1000199 [Color figure can be viewed at wileyonlinelibrary.com]

```
Diffstat
-rw-r--r-- kernel/events/hw_breakpoint.c 9
1 files changed, 7 insertions, 2 deletions

diff --git a/kernel/events/hw_breakpoint.c b/kernel/events/hw_breakpoint.c
index a556aba223da..0c82663395f7 100644
--- a/kernel/events/hw_breakpoint.c
+++ b/kernel/events/hw_breakpoint.c
@@ -456,7 +456,9 @@ register_user_hw_breakpoint(struct perf_event_attr *attr,
}
EXPORT_SYMBOL_GPL(register_user_hw_breakpoint);

-static int __modify_user_hw_breakpoint(struct perf_event *bp, struct perf_event_attr *attr)
+static int
+modify_user_hw_breakpoint_check(struct perf_event *bp, struct perf_event_attr *attr,
+                               bool check)
+{
    u64 old_addr = bp->attr.bp_addr;
    u64 old_len = bp->attr.bp_len;
@@ -468,6 +470,9 @@ static int __modify_user_hw_breakpoint(struct perf_event *bp, struct perf_event_
    bp->attr.bp_type = attr->bp_type;
    bp->attr.bp_len = attr->bp_len;

+    if (check && memcmp(&bp->attr, attr, sizeof(*attr)))
+        return -EINVAL;
+
    err = validate_hw_breakpoint(bp);
    if (!err && modify)
        err = modify_bp_slot(bp, old_type);
@@ -505,7 +510,7 @@ int modify_user_hw_breakpoint(struct perf_event *bp, struct perf_event_attr *att
    else
        perf_event_disable(bp);

-    err = __modify_user_hw_breakpoint(bp, attr);
+    err = modify_user_hw_breakpoint_check(bp, attr, false);

    if (err) {
        if (!bp->attr.disabled)
```

symbol shows the deletion in the code and “@@” shows the starting and ending lines of changes to make. There are changes made in this patch file at three different places. Furthermore, patch files also includes the address of the old file (before patch applied) and the new file (after patch applied), which are also shown in figure with - - - **a** and + + + **b**, respectively. So, our main concerned tag was - - - **a**, which is considered as a vulnerable file.

A reliable vulnerable dataset is a prerequisite for the vulnerability detection accurately. To collect the vulnerable code, we leveraged the CVE¹¹ platform, which has a list of CVE including the description, public reference and an identification number. As the CVE site displays different public references, from where we can explore or download the committed code against each CVE. In Figure 2, the diffstat of CVE-2018-1000199 is shown in the link,

- - - **a**/kernel/events/hw_breakpoint.c

shows the vulnerable file location, that is, before the patch applied on it. During crawling different sources, we search for the commit commands, which shows the full description of that specific vulnerability and its patch file. These commit references are from different sources (sites), so quite hard to get the source code of these patch files. To solve this issue, we selected some top reference site, that is, Git repository, Github, BugZila, and so forth.

Filter irrelevant commits: As we have mentioned earlier that we wrote a web crawler to download vulnerable files from different web sources. The reference links on the CVE site are for the convenience of user to help him to distinguish between vulnerabilities. Some of these references are just informative, thus we ignore these references because there are no commits to see the patch files or to download the vulnerable file. In our case, we downloaded the whole list of CVE entries of about 111,500 from CVE site. By exploring the list, we came to know that, there are many CVE entries which do not provide the references to their patch files. So, it was difficult to get their vulnerable source code. From the list, we just selected some references sites, on which we able to get the patch files and then locate its unpatched source file. To get the big collection of venerable source code, we wrote different web crawlers for different references sites. A few references sites includes, Kernel.org git repositories¹², GitHub repository¹³, Bugzilla¹⁴, Openssl¹⁵, and so forth. Vulnerability dataset consists on hundreds of vulnerabilities source code files, which were extracted from a very reliable source (cve.mitre.org). This dataset consists on already publicly known cybersecurity vulnerabilities. Tables 1 and 2, shows the downloaded dataset of open source projects of 7-TB C/C++ source code and the vulnerable code dataset against its CVE-number, respectively.

¹¹<https://cve.mitre.org/index.html>

¹²<https://git.kernel.org/>

¹³<https://github.com/>

¹⁴<https://www.bugzilla.org/>

¹⁵<https://git.openssl.org>

Language	size	No of projects	No of files (millions)	LOC (billions)
C & C++	7 TB	152,823	235	83

TABLE 1 OSS dataset collection

Source	No of CVE's	Number of files	LOC
git.kernel.org	1168	1999	2,256,833
git.videolan.org	194	202	227,802
cgit.freedesktop.org	131	225	239,009
git.qemu.org	123	138	189,458
git.savannah.gnu.org	91	116	178,409
git.openssl.org	85	130	149,565
git.ghostscript.com	79	121	123,510
Total	1871	2931	3,364,586

TABLE 2 Vulnerability dataset collection (May-2019)

Abbreviation: OSS, open source software.

4 | DESIGN AND IMPLEMENTATION OF SQVDT

In this section, we describe the design and implementation of our proposed approach SQVDT for vulnerability detection at source code level. The core idea behind SQVDT is to detect the vulnerable source code in any software, to test the security. Our goal is to detect vulnerable source code in the subject system against vulnerability dataset by using code clone detection technique, where we already have the fingerprint signature-set of vulnerable source code dataset. Our developed vulnerability detection method works at file level granularity, in which we detect similar source code files. The main architecture is shown in Figure 3, where on the left side of the figure, we take vulnerable code dataset and extract different features from it to build its fingerprint, and on the right side of the figure, we take a subject system (test project) to detect and locate vulnerable source code by applying different filters on it.

This section has been further divided into the following phases.

4.1 | Preprocessing on source code (Phase:1)

This is the first phase of SQVDT design, in which we extracted the **vulnerable code from the patch files** by tracing its original source code file (before patch applied), then remove all the unwanted code, that is, comments, spaces and so forth, from that file. In the final step of preprocessing, we convert source code into specific units (tokens) as shown in Figure 4. In this figure, the top side shows the path and the changes made in vulnerable file, the left side of the down part shows the patch file, the middle part shows the vulnerable code fragment, and the right side shows the preprocessed code. The purpose of this phase is to clean the source code and prepare it for detection process. During preprocessing, we used lexical analyzer technique for tokenization by assigning specific id's to each token as shown in Figure 4. The lexical analyzer performed several tasks on the source code, that is, ignore the spaces and comments; replace identifiers by an id plus (+) their occurrence number; replace all the unified strings to "empty"; integer values by 0; floating-point by 0; Boolean values by true; and so forth. as shown in Figure 4. For example, this is an expression $x = a + y * z$; then the lexical analysis become $id0 = id1 + id2 * id3$;

Identifier rename handling: During our analysis of vulnerability source code, we found that some developers use the same source code file but made a minor modifications in the source code by renaming some *Identifiers* in it. For example, some hackers and malware developers have performed identifier renaming to increase the survivability of their code by making it hard to detect. Therefore, to overcome this problem, our preprocessing method performed well by assigning specific ids to replace all methods name, identifiers, and so forth, without considering their actual names. We left the return statements as it is to preserve it during detection process. Even though, some class names were renamed during vulnerable code generation, we replace all the class names by assigning specific ids to it, so that they remain same in every file as shown on the right side of Figure 4.

Vulnerable Dataset



Download CVE List
(allitems.txt)

Retrieve Commit References

Patch file

Vulnerable
source files

```
{
  id0 id1 = id2 == null || id3 == 0;
  id0 id1, id2;
  id0 id1, 0, 0; id0 id1, 0, 0;
  id0 = id1;
  id0 = id1;
  id0 = id1 id2 : id3;
  if id0 {
    .....
```

1. Preprocessing & Normalization
Loading, Transforming,
Tokenizing, Chunking

SQVDT (Architecture)



2. Feature Extraction
MD5 hash, FNV hash
Sim hash, File path,
Lines of code

Fingerprint

Indexing
DB (HBase)

```
66022 origlnpath:hdfs://192.168...:9000/sourcecode/12345678911111/linux-2.6.33-lpc313x-master/drivers/hid/hid-sunplus.c clonepath:/media/hadoop/
bf443e3e-dfe2-4c66-84fd-ef5de2b7e39f/c_code/1472246179624/bug20-2.6.31-onap-master/drivers/hid/hid-cherry.c similarity:0.611881106376648
66023 origlnpath:hdfs://192.168...:9000/sourcecode/12345678911111/linux-2.6.33-lpc313x-master/drivers/hid/hid-sunplus.c clonepath:/media/hadoop/
bf443e3e-dfe2-4c66-84fd-ef5de2b7e39f/c_code/1474284409875/bcn963xx_4.12L_066_consumer-master/kernel/linux/drivers/hid/hid-cherry.c similarity:
0.611889106376648
66024 origlnpath:hdfs://192.168...:9000/sourcecode/12345678911111/linux-2.6.33-lpc313x-master/drivers/usb/host/xhci-hcd.c clonepath:/media/
hadoop/bf443e3e-dfe2-4c66-84fd-ef5de2b7e39f/c_code/1471022853153/android-ndk-tegra-nv-3.1.10-rel-15r7-master/drivers/usb/host/xhci.c similarity:
0.843206440582275
```

Vulnerability Detection

Subject System

1. Preprocessing & Normalization
2. Feature Extraction

MapReduce()

- Filter-1: Compare MD5 hashes of prefix and suffix
- Filter-2: Compare total number of tokens in a file (Units)
- Filter-3: Compare 100 FNV hashes in HBase (Elements)
- Filter-4: Retrieve files according to Minhash comparison
- Filter-5: Retrieve exact clone files according to SSH file division from all nodes in the cluster

Comparison

Updating

Results

- Sorting
- Storing
- Displaying

FIGURE 3 SQVDT main architecture. SQVDT, scalable quantitative vulnerability detection technique [Color figure can be viewed at wileyonlinelibrary.com]

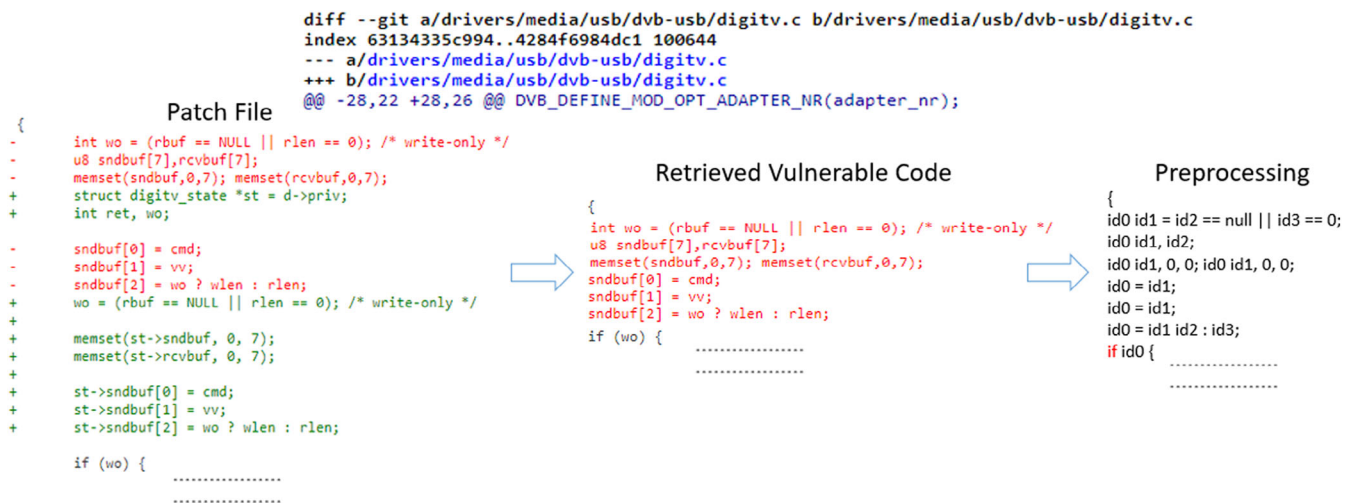


FIGURE 4 Vulnerability code extraction and preprocessing [Color figure can be viewed at wileyonlinelibrary.com]

4.2 | Feature extraction from source code (Phase:2)

This phase is the core phase of our vulnerability detection technique because these extracted features are the fingerprint, which help us to perform comparison between the subject system and the vulnerability code dataset. By extracting features, we basically reduce the amount of code for comparison purposes. This phase consists of different steps, some of them are performed in a pipeline and some of them in parallel. The extracted features are stored into HBase as a fingerprint of vulnerability dataset, that is, Row_key, Origin_id, Units, Elements and All. There are different steps performed for feature extraction.

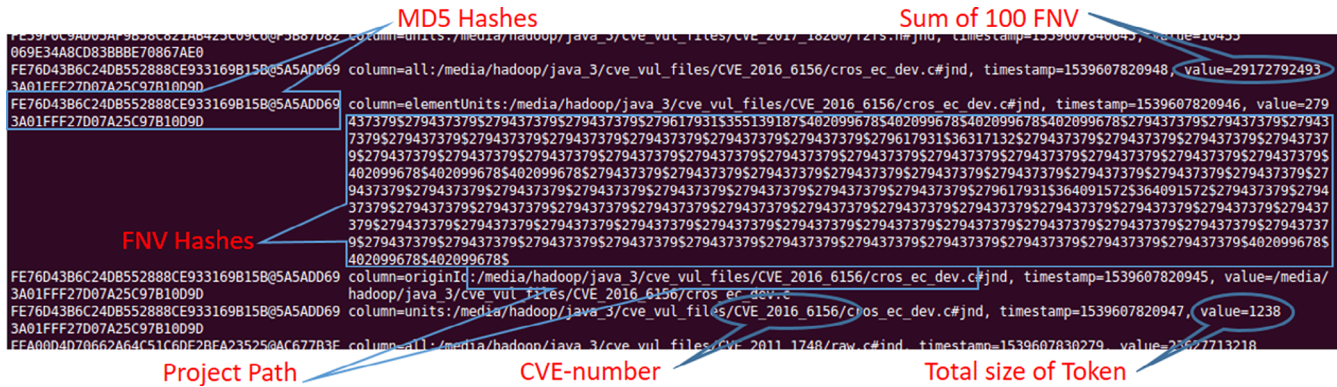


FIGURE 5 Fingerprint indexing information in HBase [Color figure can be viewed at wileyonlinelibrary.com]

MD5 hash extraction: In this step, we use MD5 hashing algorithm to extract basic feature from the source code file to differentiate every file as a unique file. Using MD5 hash, we encrypt and generate the hash value of the prefix 15 tokens and suffix 15 tokens of every file. The purpose to use 15 tokens prefix and suffix is to define a unique fingerprint of every source code file. These two hashes H1 and H2 further combined with the \$ sign and saved into HBase big-table as a Row_key, which later on help us in comparison of the source code files during vulnerability detection.

FNV hash extraction: In this step, we extract another feature for fingerprint indexing, which is FNV (Fowler-Noll-Vo)¹⁶. The reason we use FNV as another feature, because it is fast and it hashes the large amount of code with a very small conflict. we divide total tokens of each file into 10 equal parts, that is, p0, p1, p2, ... p9. Then we apply FNV hash on each division to generate FNV hash values. These hashes, later on help us in comparison and detection of the similarity ratio between the files. After generating the FNV hashes, we just select 10 smallest hashes from each division of each file. By doing this, we collected almost 100 hashes in total then add all of them into a single value (long integer) to form another feature, which shows the full characteristics of each file.

4.3 | Fingerprint indexing (Phase:3)

This section describes the fingerprint indexing part of SQVDT. The indexing performed on the vulnerability source code against its CVE number. It allows to find all the vulnerable files against a single file or for an entire system. Furthermore, we can update the fingerprint indexing information, when files are removed, added, or modified. Fingerprint indexing on extracted features is the main data structure in our approach. During indexing some functions run in parallel, for example, some source files are indexed fully independent. The indexing takes less time for large files as compared with small size files on the basis of LOC, because small files contain less code as compare to large files and have to tackled (preprocessing and feature extraction) one by one. All the fingerprint indexing information stored in HBase big-table. The proposed indexing method is very flexible, fast and easy to maintain. We can update, delete or edit any indexing record of any file at any time. Meanwhile, we can track the information of any file in less than a microsecond. The reason we selected HBase¹⁹ for indexing data storage, because it is an open source, larger production environment, support the replication between clusters, better performance and optimized for scanning index. The indexing information includes MD5 hash values, the path of the vulnerable file, FNV hashes and total token size of every source file stored in HBase. The indexing information contains all these tuples of every vulnerable file as a fingerprint. A sample of fingerprint indexing information is shown in Figure 5. Left side of this figure shows the MD5 hash values (1st feature), some other features have been labeled in red color. The following are the indexing labels, which describes the entities in the HBase table.

Row_key: MD5 hash value of prefix (H1) and suffix (H2).

Origin_id: The file path or location of a file.

Element: 100 FNV hashes of a file.

Units: Size of tokens inside a concerned file.

All: Sum of 100 FNV hashes.

¹⁶<http://www.isthe.com/chongo/tech/comp/fnv/index.html>

4.4 | Vulnerability detection (Phase:4)

This is the final phase, in which we detect, retrieve and display the vulnerable source code files. All of the source code files which meets the defined filter (Filter-1,2,3,4) conditions, retrieved through MapReduce() as shown on the right side of Figure 3. The mapper apply these filters on the subject system to sort out and calculate the fraction of the vulnerable files in the indexing table. All the retrieved cloning files display against every single subject file, which have the similar features. MapReduce() divides the task into small tasks and assign them to each map node to run. For example, a piece of file corresponds to a task and MapReduce() allocate and distribute it to the local systems for the sake of saving the communication overhead in the cluster.

When user upload a subject system to SQVDT, it identifies the subject system, including type (language) of files inside that project folder, size, and so forth. Then MapReduce() assigned this subject system equally to each node in the cluster for vulnerability detection. All the filtered valid file paths of subject system stored in HDFS, which makes it convenient for the MapReduce to process. All the concerned features from subject system extracted by applying different filters on it, that is, MD5 hash values, chunks *Units* calculation, FNV hashes, Minhash and other value operation. The filters compare the values of subject system to the indexing values of fingerprint. When features of subject system and the fingerprint dataset becomes 100% similar, we locate it, retrieve it, and display it as a vulnerability. We use MapReduce rule of divide and conquer to count the similar features between files. MapReduce fast counting function is convenient to implement the vulnerability detection through distributed and parallel computing, especially the mapping process. The input file path is the path of the subject system, and the output context shows the intermediate results of the map program. The whole process detection consists on the following steps and filters.

Preparing the subject system: Through the HDFS file system, we uploaded the subject system (test project) in the Hadoop environment for vulnerability detection. Then the preprocessing and normalization performed on this subject system to clean the source code as shown in Figure 4. After cleaning the source code, we create chunks and generate their hashes to compare them with the same hashes which were saved as a fingerprint, including MD5 hashes, FNV hashes, Min-hashes and the file path. After preparing the source code we perform the following filters.

Filter 1: In this filter, the detector detects vulnerable files from subject system by comparing their hash values saved in HBase indexing table, which have the same Row_key (M1@M2) values, where M1 is the hash value of prefix chunk and M2 is the hash value of suffix chunk of a file. In the case of hash matches, we retrieve those results to apply next filter.

Filter 2: In this filter, we get more abstract cloning files, which are the indication of vulnerability. We apply this filter on the previously retrieved files. The detector counts total number of tokens in every file of the subject system and then compare them into the stored fingerprint index (Units) in HBase.

Filter 3: In this filter, we compare 100 FNV hashes (Elements) in the files of subject system to the fingerprint. This *Element* entity, was hashed from each file by dividing it into 10 equal parts during fingerprint indexing of vulnerability dataset.

Filter 4: After getting the abstract cloning files on every filter, we further evaluate the results for *exact clones* by using Minhash. The Minhash algorithm help us in further comparison between resultant files to find the exact clone files.

Filter 5: This is the last filter of detection process, in which the detector uses SSH protocol (Secure Shell) to send the subject system files to every attached machine in the cluster. These files of subject system divide into chunks, generating their MD5 hashes and perform the final comparison task. After getting the results, the detector evaluate them, combine them and display them to the user. Meanwhile, it saves the results as a record for further evaluation and use.

5 | EXPERIMENTS AND RESULTS

For experimental purposes and to prove the speed, effectiveness, reliability, scalability, and performance of our approach, we detect vulnerabilities in two ways.

5.1 | OSS dataset as a fingerprint

In this experimental way, we build fingerprint index of millions of OSS, consisting on billions of lines of code of C/C++. After building an index of almost 7-TB source code (Table 1) at large-scale level, then we took vulnerable source code files

```

15237 originpath:hdfs://192.168...:9000/CVE_code/linux_and_kernel/CVE-2018-17182/root/include/linux/vmcache.h
15238 clonepath:/media/Code2/c_code/1474977326794/linux-master/include/linux/vmcache.h similarity: 1.0
15239 originpath:hdfs://192.168...:9000/CVE_code/linux_and_kernel/CVE-2018-6412/root/drivers/video/fbdev/sbuslib.c
15240 clonepath:/media/Code2/c_code/1474989698583/linux-xtensa-xtensa-ftrace/drivers/video/fbdev/sbuslib.c similarity: 1.0
15241 originpath:hdfs://192.168...:9000/CVE_code/linux_and_kernel/CVE-2018-6412/root/drivers/video/fbdev/sbuslib.c
15242 clonepath:/media/Code2/c_code/1475110236428/net-next-nuse-for-asm-upstream-v3/drivers/video/fbdev/sbuslib.c similarity: 1.0
15243 originpath:hdfs://192.168...:9000/CVE_code/linux_and_kernel/CVE-2018-13406/root/drivers/video/fbdev/uvesafb.c
15244 clonepath:/media/Code2/c_code/1474943052043/Explay_A350_kernel_source_code-master/drivers/video/uvesafb.c similarity: 1.0
15245 originpath:hdfs://192.168...:9000/CVE_code/linux_and_kernel/CVE-2018-10689/root/btt/devmap.c
15246 clonepath:/media/Code2/c_code/1477340503128/s-blktrace-master/blktrace-userspace/btt/devmap.c similarity: 1.0
15247 originpath:hdfs://192.168...:9000/CVE_code/git_qemu/CVE-2017-6505/qemu.git/hw/usb/hcd-ehci.c
15248 clonepath:/media/Code2/c_code/1476838765540/qemu-xtensa-xtensa-new-ops/hw/usb/hcd-ehci.c similarity: 1.0
15249 originpath:hdfs://192.168...:9000/CVE_code/linux_and_kernel/CVE-2017-2647/root/net/ceph/crypto.c
15250 clonepath:/media/Code2/c_code/1474468871146/bcache-dev-bcache-dev/net/ceph/crypto.c similarity: 1.0
15251 originpath:hdfs://192.168...:9000/CVE_code/linux_and_kernel/CVE-2018-11506/root/drivers/scsi/sr_ioctl.c
15252 clonepath:/media/Code2/c_code/1477003306399/opw-staging-vinc-devel-configs/drivers/scsi/sr_ioctl.c similarity: 1.0
15253 originpath:hdfs://192.168...:9000/CVE_code/free_Desktop/CVE-2016-7947/root/src/XrtrConfig.c
15254 clonepath:/media/Code2/c_code/147278011339/xsrc-3.4.0/external/mit/libXrandr/dist/src/XrtrConfig.c similarity: 1.0
15255 originpath:hdfs://192.168...:9000/CVE_code/free_Desktop/CVE-2016-9807/root/gst/flux/fmt.h
15256 clonepath:/media/Code2/c_code/1476549929668/gst-combined-master/gst-plugins-good/gst/flux/fmt.h similarity: 1.0
15257 originpath:hdfs://192.168...:9000/CVE_code/free_Desktop/CVE-2016-9807/root/gst/flux/fmt.h
15258 clonepath:/media/Code2/c_code/1472225988992/fread-ink-master/graphics/x-server/packages/xpm-3.5.8/src/CrDrFrI.c similarity: 1.0
15259 originpath:hdfs://192.168...:9000/CVE_code/linux_and_kernel/CVE-2016-7425/root/drivers/scsi/arcmsr/arcmsr_hba.c
15260 clonepath:/media/Code2/c_code/1475330638026/ntrdma-pu/drivers/scsi/arcmsr/arcmsr_hba.c similarity: 1.0
15261 originpath:hdfs://192.168...:9000/CVE_code/git_qemu/CVE-2016-10155/qemu.git/hw/watchdog/wdt_i6300esb.c
15262 clonepath:/media/Code2/c_code/1475841037460/qemu-stable-2.6/hw/watchdog/wdt_i6300esb.c similarity: 1.0
15263 originpath:hdfs://192.168...:9000/CVE_code/git_qemu/CVE-2016-10028/qemu.git/hw/display/virtio-gpu-3d.c
15264 clonepath:/media/Code2/c_code/1475054155364/qemu-gnuarmeclipse/hw/display/virtio-gpu-3d.c similarity: 1.0

```

FIGURE 6 Vulnerable files detected in 7-TB of C/C++ source code (152,823 open source projects) [Color figure can be viewed at wileyonlinelibrary.com]

```

diff --git a/hw/usb/hcd-ehci.c b/hw/usb/hcd-ehci.c
index 4e2161b..d07f228 100644 (file)
--- a/hw/usb/hcd-ehci.c
+++ b/hw/usb/hcd-ehci.c
@@ -1389,7 +1389,7 @@ static int ehci_process_itd(EHCIState *ehci,
{
    USBDevice *dev;
    USBEndpoint *ep;
-   uint32_t i, len, pid, dir, devaddr, endp;
+   uint32_t i, len, pid, dir, devaddr, endp, xfers = 0;
    uint32_t pg, off, ptr1, ptr2, max, mult;

    ehci->periodic_sched_active = PERIODIC_ACTIVE;
@@ -1479,9 +1479,10 @@ static int ehci_process_itd(EHCIState *ehci,
    ehci_raise_irq(ehci, USBSTS_INT);
}
itd->transact[i] &= ~ITD_XACT_ACTIVE;
+   xfers++;
}
}
return 0;
+   return xfers ? 0 : -1;
}

```

FIGURE 7 CVE-2015-8558: Published in QEMU and also found in TQSIM simulator [Color figure can be viewed at wileyonlinelibrary.com]

(Table 2) as a subject system and detect vulnerabilities in these OSS. The results were saved in a HDFS file including the following information.

- Path of the subject files
- Path of the cloned files
- Similarity degree between two files

More than 15,000 results were retrieved from almost 152,823 of open source projects against almost 3000 vulnerable source code files as shown in Figure 6. Most of the results were concerned about different versions of Linux. A few results (projects) are discussed in this article, which have not been announced vulnerable before.

CVE-2015-8558: This CVE allows local user (guest) OS administrator to cause a DOS attack to initiate an infinite loops and CPU consumption, via isochronous transfer descriptor list. The patch file of this CVE is shown in Figure 7. This CVE has the CVSS score of 4.9, and it was found in `/hw/usb/hcd - ehci.c` in *QEMU*. We found the same CVE vulnerable files in *TQSIM* (Timed QEMU-based Simulator)¹⁷, which is an open source cycle-approximate simulation tool that is built on *QEMU*. *TQSIM* is developed by CAPLab laboratory, in Seoul National University, and it is sponsored by the SAIT (Samsung Advanced Institute of Technology). In this simulation tool, this vulnerable file still exist and the patch file has not been applied. People are still using this tool without knowing that this is vulnerable because its information has not been published anywhere. The same vulnerability has been detected in *MBA* (Malware Behavior Analyzer)¹⁸, which is a *QEMU*-based sandbox system, dedicated to malware analysis.

¹⁷<https://github.com/cap-lab/tqsim>

¹⁸<https://github.com/GlacierW/MBA/tree/master/hw/usb>

CVE-2010-2497: This CVE was discovered in *FreeType* (versions before 2.4.0), with CVSS score of 6.8. It allows the remote users (attackers) to perform a DOS attack (application crash), or execute the arbitrary code. The remote user is not required any authentication to exploit the vulnerability. The same vulnerability detected in *RT-Thread*, which is an open source operating system (real-time) for embedded devices. It is under an Apache 2.0 license. This CVE was announced and patched in *FreeType* in year 2010 but the vulnerable files are not patched in *RT-Thread*. Thus, *RT-Thread* is still vulnerable and can be exploited by an attacker very easily. The same vulnerability is found in *KA-Lite-Wrapper*¹⁹ for Android, which is an offline version to encapsulate the *Django* project.

CVE-2018-7728, CVE-2018-7729, CVE-2018-7730, CVE-2017-18233, CVE-2017-18234: These five CVEs were discovered in *Exempi* (version 2.4.4). These CVEs have same CVSS score of 4.3 and lead to a heap-based vulnerability of buffer overread. All these vulnerabilities do not require any authentication to exploit it. Through our vulnerability detection approach, we have detected all of these five vulnerabilities in an other OSS named *ImageWalker*²⁰ (version 2.31), which is an image organizing tool to help you manage your photos, including viewing, cataloguing, publishing, web page creations, thumbnail generation, printing, image conversion, slide showing, and screen saver. These vulnerabilities were not announced in *ImageWalker*, that's why the files are still not patched yet.

CVE-2012-3552: This vulnerability is a race condition of the IP implementation in *Linux-Kernel* before version 3.0, which allows an attacker to cause a DOS attack by sending packets during handling a network traffic without any authentication. We detected the same vulnerability in *Htc-Kernel-Endeavoru*²¹, which is not announced yet.

CVE-2010-2808: This CVE was discovered in year 2010, in *FreeType* (versions before 2.4.2), with CVSS score of 6.8. *FreeType* is a software library to render fonts. It allows the remote users to cause DOS attack. We found the same vulnerable file in *APV* (Android Pdf Viewer), including encryption function.

CVE-2017-2625: This vulnerability has been discovered in *libXdmcp* before versions 1.1.2, which included weak entropies to generate the session keys. Through this vulnerability, an attacker could use the information from the process list by allowing them to hijack the other sessions. We detected the same vulnerable files in *MINIX*, which is a free and open source operating system designed to be flexible and secure. It is based on a microkernel architecture.

CVE-2015-8961: This vulnerability was first time announced in year 2015 in *Linux-kernel* before version 4.3.3 and it has a very critical CVSS score of 9.3. By exploring a patch file of this vulnerability, we can see that a function in the file *fs/ext4/ext4_jbd2.c* allows a local users to gain fully privileges and cause a DOS attack by leveraging an improper access to a specific error field. The same vulnerable file we discovered in *Xiaomi-kernel*²².

Now we will discuss some of the vulnerable projects available on GitHub which share the same vulnerability of single CVE-ID. Even though the patch files of these vulnerabilities are already published but these projects are still not patched and people are using them without knowing this. The following CVEs are some of these vulnerabilities discovered in different open source projects. The results were too many to mention in this article. Thus, we uploaded the following few of the results on GitHub²³ as a reference against its detected CVE-number.

CVE-2013-0310: This DOS (pointer dereference and system crash) vulnerability was first discovered in *Linux kernel* before 3.4.8, in the function of *cipso_v4_validate* in *net/ipv4/cipso_ipv4.c* file. We detected this vulnerability in many other OSS available on GitHub. There is a list of 42 projects that contain this vulnerability (see footnote 23).

CVE-2018-6412: This vulnerability was found in the *Linux kernel* 4.15 that exist in a function *sbusfb_ioctl_helper()* of file *drivers/video/fbdev/sbuslib.c*. This is an information leakage vulnerability that allows an attacker to steal information. Its patch is straight-forward, we should change "int index" to "unsigned int index" in two switch-cases to overcome on this vulnerability. There is a list of 23 projects that contain this vulnerability (see footnote 23).

CVE-2017-11472: This is another "obtain information" vulnerability that allow local user to obtain sensitive information from memory. This vulnerability is in a function *acpi_ns_terminate()* of a file *drivers/acpi/acpica/nsutils.c*. This vulnerability causes a threat because the old kernel before version 4.9 shows the memory locations of functions in a stack dump. To fix this operand leak for enhancing security, a patch is published to modify this function. There is a list of 16 projects that contain this vulnerability (see footnote 23).

CVE-2016-10318: This is a DOS vulnerability discovered in year 2016, that allow users to assign an encryption policy to a directory owned by a different user. There is a missing authorization check in function *fsencrypt_process_policy()* of a file *fs/crypto/policy.c*. On an *ext4* or *f2fs* file system with a file encryption supported, a user can set an encryption

¹⁹<https://github.com/66eli77/wch-android-kalite>

²⁰<https://github.com/ZacWalk/ImageWalker>

²¹<https://github.com/TeamNDVRu/htc-kernel-endeavoru>

²²https://github.com/MiCode/Xiaomi_Kernel_OpenSource/blob/dipper-o-oss/fs/ext4

²³https://github.com/znd15/Vul_in_7TB



FIGURE 8 CVE-2006-1368 patch file in the Linux kernel before 2.6.16 [Color figure can be viewed at wileyonlinelibrary.com]

FIGURE 9 CVE-2006-1368 CVSS scores and vulnerability types [Color figure can be viewed at wileyonlinelibrary.com]

policy on any empty directory (*), on which they have read-only access. It is an obviously problematic, because such a directory may be owned by an another user. The new encryption policy prevent that specific user from creating files in their directory. We can fix this vulnerability by requiring inode_owner_or_capable() permission by setting an encryption policy. This means that the caller must own the file, or the caller must have that capability. There is a list of 38 projects that contain this vulnerability (see footnote 23).

CVE-2017-18075: This DOS (kfree of an incorrect pointer or unspecified impact by executing a sequence of system calls) vulnerability exist in a file crypto/pcrypt.c of Linux kernel before version 4.14.13. The pcrpt file is using an old way of freeing instances, where the free() method specified in “struct crypto_template” pass a pointer to the “struct crypto_instance.” But the crypto_instance is kfree()’d, which is not correct because the memory was allocated as aead_instance, that contains the crypto_instance at nonzero offset. Thus, wrong pointer was generated kfree()’d. This vulnerability can be fixed by switching to the new way of free aead_instance where the free() method is specified in aead_instance by itself. There is a list of 33 projects that contain this vulnerability (see footnote 23).

One of the vulnerability (CVE-2006-1368) patch file is shown in Figure 8, which is a buffer overflow vulnerability in the Linux-kernel before version 2.6.16 that allows an attackers to cause denial-of-service. This vulnerability is very critical with high risk as shown in Figure 9. This vulnerability was detected in GPL source dump for the Cisco-WAP4410N²⁴. Another very critical vulnerability (CVE-2015-8961) was discovered in Xiaomi mobile-phone-Kernel²⁵. The patch file of this vulnerability is available on this link²⁶. In this vulnerability, the ext4 journal stop function in fs/ext4/ext4_jbd2.c in Linux kernel before version 4.3.3, which allows users to gain privileges and cause a DOS attack. In this vulnerability only one file was changed and patched, so we are sure to say that this vulnerability exist in Xiaomi_Kernel. In this section, we further display some detected vulnerabilities by SQVDT in different software. Table 3, shows a few results of discovered vulnerabilities in 7-TB of source code including their CVE-ID, CVSS_score, Vulnerability_type, CWE-ID, and number of Occurrence. In total, we retrieved almost fifteen thousand vulnerable files of almost 504 CVE-IDs from 152,823 open source projects including their different versions also. Table 4 displays the list of some OSS, in which vulnerabilities are discovered with respect to different CVSS-score. In total almost 15,245 vulnerability results retrieved from thousands of

²⁴https://github.com/cjdelisle/wap4410n_v2.0.7.4_gpl

²⁵https://github.com/MiCode/Xiaomi_Kernel_OpenSource

²⁶<https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=6934da9238da947628be83635e365df41064b09b>

TABLE 3 50 most occurred discovered vulnerabilities in 7-TB of source code

CVE-ID	CVSS_score	Vulnerability_type	CWE-ID	Occurrence
CVE-2013-7421	2.1	–	264	2522
CVE-2011-2918	4.9	DoS overflow	399	715
CVE-2014-9644	2.1	–	264	557
CVE-2010-0291	4.6	DoS +Gain privileges	264	537
CVE-2016-7097	3.6	Gain privileges	285	307
CVE-2010-3081	7.2	Overflow +Gain privileges	119	236
CVE-2014-7822	7.2	DoS	264	231
CVE-2015-2150	4.9	DoS	264	141
CVE-2011-4324	4.9	DoS	–	131
CVE-2017-9059	4.9	DoS	399	130
CVE-2010-2943	7.9	Obtain information	200	127
CVE-2016-4565	7.2	DoS	264	123
CVE-2016-6213	4.7	DoS	400	103
CVE-2011-4914	6.4	DoS Obtain information	20	100
CVE-2014-3184	4.7	DoS overflow	119	100
CVE-2014-0181	2.1	Bypass	264	97
CVE-2012-3552	5.4	DoS	362	92
CVE-2015-8948	5	Obtain information	125	91
CVE-2014-9710	6.9	Gain privileges + Bypass	362	85
CVE-2013-7271	4.9	Obtain information	20	84
CVE-2013-7266	4.9	Obtain information	20	84
CVE-2013-7267	4.9	Obtain information	20	84
CVE-2013-7268	4.9	Obtain information	20	84
CVE-2013-7269	4.9	Obtain information	20	84
CVE-2013-7270	4.9	Obtain information	20	84
CVE-2016-9807	4.3	DoS	125	80
CVE-2012-5374	4	DoS	310	70
CVE-2012-5375	4	DoS	310	70
CVE-2017-9074	7.2	DoS	125	69
CVE-2013-6282	7.2	Read or modify	20	69
CVE-2009-1961	1.9	DoS	362	69
CVE-2016-1576	7.2	Gain privileges	264	67
CVE-2016-1575	7.2	Gain privileges	264	67
CVE-2017-5967	2.1	Obtain information	200	63
CVE-2014-5471	4	DoS	399	57
CVE-2014-5472	4	DoS	20	57
CVE-2017-2647	7.2	DoS + Gain privileges	476	56
CVE-2015-8839	1.9	DoS	362	54
CVE-2015-8961	9.3	DoS + Gain privileges	416	50
CVE-2009-1439	7.8	DoS + Overflow	119	50
CVE-2012-0044	7.2	DoS overflow + Gain privileges	189	50
CVE-2017-17805	7.2	DoS	20	50
CVE-2012-1097	7.2	DoS	–	50
CVE-2016-9794	7.2	DoS	416	50
CVE-2016-10153	7.2	DoS + Memory corruption	399	50
CVE-2018-17182	7.2	Overflow + Gain privileges	416	50
CVE-2010-3904	7.2	Gain privileges	20	50
CVE-2016-2143	6.9	DoS	20	50
CVE-2016-9777	6.9	DoS + Gain privileges	125	50
CVE-2011-3188	6.8	DoS	–	50

TABLE 4 List of some open source software, in which vulnerabilities are discovered with respect to different CVSS-score

Web-URL	CVE-ID	CVSS-score
https://github.com/ablight/qemu/archive/master.zip	CVE-2016-7161	10
https://github.com/felipeqc/semi/archive/master.zip	CVE-2010-2521	10
https://github.com/tartakynov/enso/archive/updatesfx.zip	CVE-2013-1591	10
https://github.com/google/capsicum-linux/archive/procdesc-4.8.zip	CVE-2016-9313	9.3
https://github.com/google/capsicum-linux/archive/misc-4.8.zip	CVE-2016-7910	9.3
https://github.com/google/ktsan/archive/tsan.zip	CVE-2015-8962	9.3
https://github.com/OSLL/qemu-xtensa/archive/windowed-2.5.zip	CVE-2016-1568	9.3
https://github.com/libos-nuse/net-next-nuse/archive/master.zip	CVE-2018-17182	7.2
https://github.com/SM-G920Pold/arter-kernel/archive/experimental.zip	CVE-2018-17182	7.2
https://github.com/USBhost/FIRE-ICE/archive/M10.5.y.zip	CVE-2017-7374	7.2
https://github.com/ljalves/linux_media/archive/latest.zip	CVE-2017-5548	7.2
https://github.com/SM-G920Pold/arter/archive/n9208.zip	CVE-2018-17182	7.2
https://github.com/MiCode/Xiaomi_Kernel_OpenSource/archive/gemini-m-oss.zip	CVE-2014-3631	7.2
https://github.com/kula85/perf-sqlite3/archive/mybranch.zip	CVE-2016-1576	7.2
https://github.com/chaseyu/f2fs-dev/archive/linux-3.4.zip	CVE-2018-17182	7.2
https://github.com/freedreno/kernel-msm/archive/stuff.zip	CVE-2018-10938	7.1
https://github.com/cmotc/fireaxe-kernel-grsec/archive/master.zip	CVE-2016-5728	5.4
https://github.com/wayland-project/wayland/archive/master.zip	CVE-2017-16612	5
https://github.com/Broadcom/cygnus-linux/archive/iproc-gpio-v3.zip	CVE-2016-5244	5
https://github.com/cmnyard/linux-live-app-coredump/archive/master.zip	CVE-2018-11232	4.9
https://github.com/OSLL/qemu-xtensa/archive/xtensa-ccount.zip	CVE-2017-5856	4.9
https://github.com/OSLL/qemu-xtensa/archive/xtensa-user.zip	CVE-2017-9060	4.9
https://github.com/google/ktsan/archive/tsan-dev.zip	CVE-2016-8646	4.9
https://github.com/ntrdma/ntrdma/archive/master.zip	CVE-2018-11232	4.9
https://github.com/android-ia/kernel_intel-uefi/archive/master.zip	CVE-2013-7268	4.9
https://github.com/dashulu/s-blktrace/archive/master.zip	CVE-2018-10689	4.3
https://github.com/davibe/gst-plugins-good/archive/calayer.zip	CVE-2016-9807	4.3
https://github.com/darnir/wget/archive/west-cookie-alone.zip	CVE-2016-4971	4.3
https://github.com/ZacWalk/ImageWalker/archive/master.zip	CVE-2017-18236	4.3
https://github.com/sebras/gst-plugins-good/archive/master.zip	CVE-2016-9807	4.3
https://github.com/ZacWalk/ImageWalker/archive/master.zip	CVE-2018-7728	4.3
https://github.com/Stone-Age-Machine/SAME/archive/dev.zip	CVE-2012-5669	4.3
https://github.com/alard/wget-lua/archive/master.zip	CVE-2016-4971	4.3
https://github.com/FMDDevs/android_kernel_lenovo_a3300/archive/master.zip	CVE-2012-5375	4
https://github.com/ntrdma/ntrdma/archive/pu.zip	CVE-2016-10318	4
https://github.com/syhost/android_kernel_zte_nx503a/archive/kk-4.4.zip	CVE-2012-5374	4
https://github.com/tarunkapadia93/gk_a6k/archive/gk.zip	CVE-2014-5471	4
https://github.com/google/capsicum-linux/archive/no-upstream-4.7.zip	CVE-2016-10318	4
https://github.com/linusw/linux-bfq/archive/master.zip	CVE-2016-7097	3.6
https://github.com/google/capsicum-linux/archive/misc-4.8.zip	CVE-2016-7097	3.6
https://github.com/boddob/linux/archive/master.zip	CVE-2018-18386	2.1
https://github.com/PatrikKT/android_kernel_huawei_y536a1/archive/cm-11.0.zip	CVE-2014-9644	2.1

Note: In total almost 15,245 results from thousands of projects, which are suspected to be have vulnerable code.

FIGURE 10 CVE-2015-8961 patch file in the Linux-kernel before 4.3.3 [Color figure can be viewed at wileyonlinelibrary.com]

```
Diffstat
-rw-r--r- fs/ext4/ext4_jbd2.c 6
1 files changed, 3 insertions, 3 deletions

diff --git a/fs/ext4/ext4_jbd2.c b/fs/ext4/ext4_jbd2.c
index d418431..e770c1ee 100644
--- a/fs/ext4/ext4_jbd2.c
+++ b/fs/ext4/ext4_jbd2.c
@@ -88,13 +88,13 @@ int __ext4_journal_stop(const char *where, unsigned int line, handle_t *handle)
     return 0;
 }

+    err = handle->h_err;
-    if (!handle->h_transaction) {
-        err = jbd2_journal_stop(handle);
-        return handle->h_err ? handle->h_err : err;
+    rc = jbd2_journal_stop(handle);
+    return err ? err : rc;
     }

     sb = handle->h_transaction->t_journal->j_private;
-    err = handle->h_err;
-    rc = jbd2_journal_stop(handle);

     if (!err)
```

projects, which are suspected to have vulnerable code. The patch file shown in Figure 10 is also very critical vulnerability, that allows user to gain privileges to perform denial-of-service attack. In this vulnerability only one file was changed and patched, so we are sure to say that this vulnerability exist in Xiaomi_kernel.

5.2 | Vulnerability dataset as a fingerprint

In this experiment of vulnerability detection, we build fingerprint index of collected vulnerability dataset and took different OSS as a subject systems, that is, *Android_kernel_htc*, *FindX-8.1-kernel*, *Linux-5.1.4*, and *Linux-5.3.7*. With the help of our proposed vulnerability detection technique (SQVDT), we detected vulnerable files in these subject systems. The results of two subject system are discussed below. In *linux-5.1.4*²⁷, we detected almost eight vulnerable files. These vulnerabilities includes *CVE-2013-7421*, *CVE-2014-3184*, *CVE-2015-3212*, *CVE-2016-1583*, and *CVE-2015-4178*. However, these vulnerabilities can also cause different attacks.

In *Android_kernel_htc*, we detected almost 70 vulnerable files with respect to vulnerability CVE-numbers. These vulnerabilities includes, *CVE-2016-2185*, *CVE-2016-10318*, *CVE-2015-2150*, *CVE-2014-5206*, *CVE-2014-5207*, *CVE-2013-7263*, *CVE-2013-7265*, *CVE-2013-7281*, *CVE-2013-7027*, *CVE-2013-7339*, and so forth. There were 71 vulnerability results detected *Android_kernel_htc*²⁸. The vulnerabilities detected in this subject system consists of different severity levels (CVSS score between 9 and 10) including critical vulnerabilities, that is, *CVE-2014-7822* and *CVE-2014-5206*. The results are published and uploaded on GitHub²⁹.

5.3 | Displaying results

As we mentioned earlier that we developed a vulnerability detection platform. In this section, we show two screenshots of our platform. Figure 11 shows the results of vulnerabilities detected in 7-TB of C/C++ source code, in the top bar, we can see a different menu, including Results, Detect, Database (building index or fingerprints), and Login. On the left side of the middle section, it displays the top three most detected vulnerable files and on the right side, it displays the top most detected CVE's. The down part of the figure displays the number of CVE's detected of different years. Meanwhile, we can search the vulnerability results by year, CVE numbers, project name, or by severity level, and so forth. Figure 12, shows two source code files, the left file is a detected cloned file and the right side is our subject system file (vulnerable). We can see that both files have almost the same source code, except include statements, empty lines, or comments.

6 | RESULTS EVALUATION AND IMPLICATIONS

As it is mentioned in previous section, that we performed two types of experiments to detection vulnerabilities at source code level, that is, OSS dataset as a fingerprint and vulnerability code dataset as a fingerprint. During using OSS as a

²⁷<https://www.kernel.org/>

²⁸https://github.com/CyanogenMod/android_kernel_htc_msm8974

²⁹https://github.com/znd15/vul_in_HTC_kernel

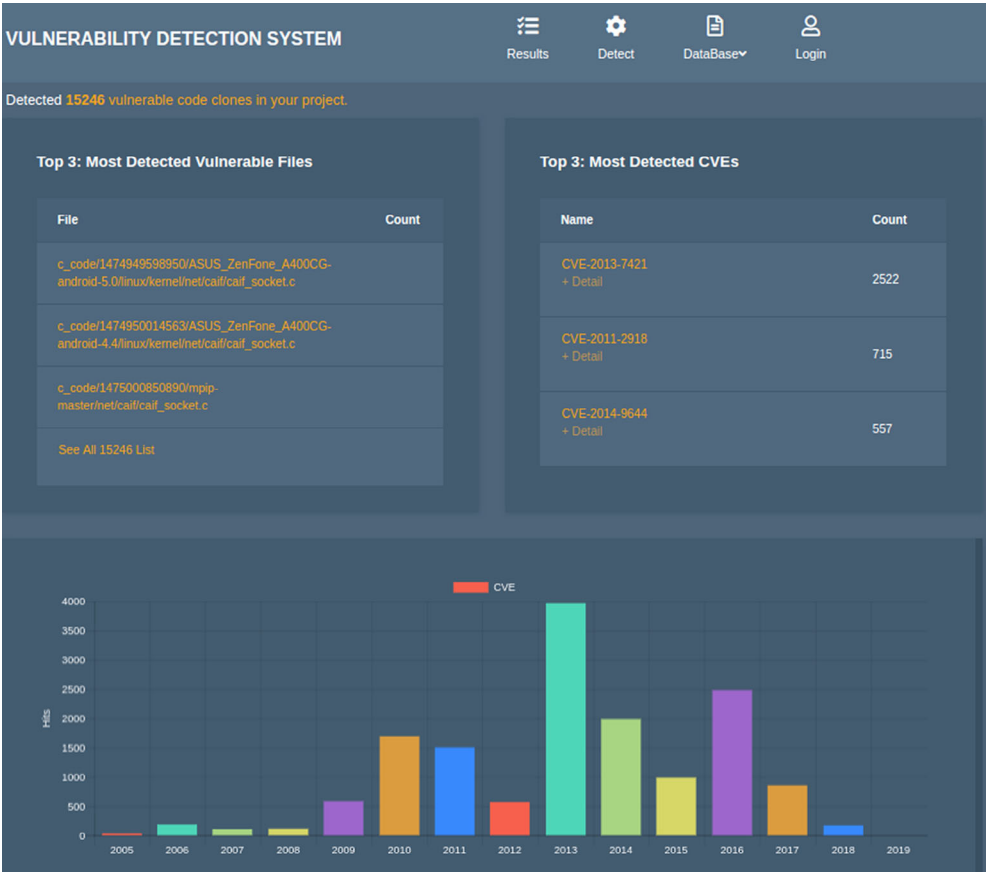


FIGURE 11 Screenshot of displaying detected vulnerabilities by year [Color figure can be viewed at [wileyonlinelibrary.com](#)]

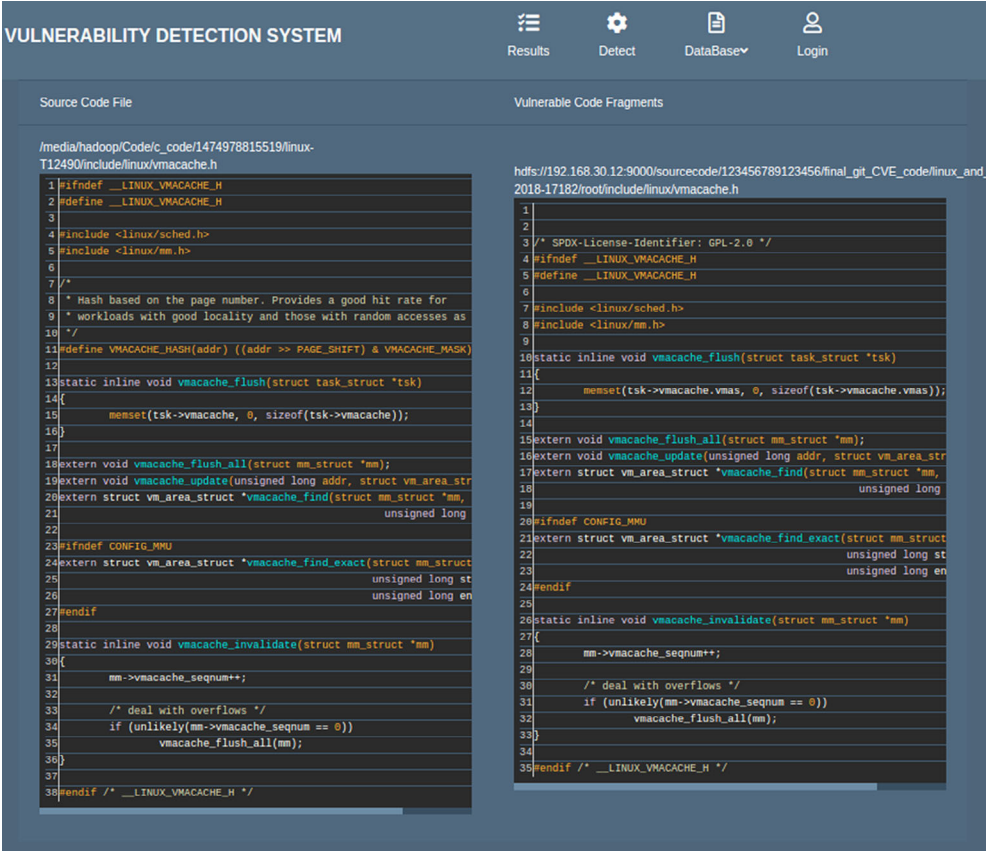


FIGURE 12 Screenshot of displaying subject file and detected vulnerable file [Color figure can be viewed at [wileyonlinelibrary.com](#)]

TABLE 5 Detected results evaluation

Subject projects	Total results	Evaluated results	True positive	False positive	Accuracy
152,823 Projects (7-TB source code)	13,580	200	99%	1%	99%
Linux-5.1.4 (Latest-version)	8	8	100%	0%	100%
Android_kernel_htc	71	50	99%	1%	99%

TABLE 6 Comparison with state-of-the-art-tools

Techniques	True positive	False positive
ReDeBug	81.0	17.6
Checkmarx	95.0	5.0
Deckard	9.5	90.0
VulPecker	84.0	15.0
VulDeePecker	93.0	5.7
VCIPR	82.3	17.8
UDDY	99.0	0.0
SQVDT	99.0	1.0

Abbreviation: SQVDT, scalable quantitative vulnerability detection technique.

fingerprint, We evaluate the *Accuracy* by using standard metrics such as *true positive (TP)*, *false positive (FP)*, and the *Accuracy*. We randomly select some of the results as the number is shown in Table 5, for the evaluation of our technique and count the *Accuracy* rate. To overcome the bias, the results were manually evaluated. Table 5 shows the *TP* rate, *FP* rate, and the *Accuracy* rate.

In the first experiment, we used vulnerability source code as a subject system and detected vulnerabilities in almost 150 thousands projects. In total 13,580 vulnerable files were detected from 7-TB of source code. From which, we evaluated almost 200 files by checking the similarity of both files. we counted almost 99% *TP* and 1% *FP* rate. From Table 5, we can see that the *Accuracy* rate is 100% for *Linux-5.1.4* subject system, in total eight results were retrieved and evaluated manually by comparing both files. In *Android_kernel_htc*, 50 results were evaluated out of 71, and the *Accuracy* rate was counted up to 99%.

From Table 5, we can see that we have very low *FP*, which indicates that our vulnerability detection technique is quite effective. As we mentioned earlier that our vulnerability detection technique is based on file level code clone detection method. So, we detected and retrieved all the vulnerable files, which have 100% similar source code. Table 5, shows the effectiveness and the *Accuracy* of our approach to detect vulnerabilities at source code level.

Furthermore, the *Accuracy* of our technique is also depend on the collection of known vulnerabilities. As long as, the extracted vulnerable code is correct, the *Accuracy* of vulnerability detection will automatically be high because we detected vulnerabilities on the basis of previously published CVEs.

Comparison with state-of-the-art-tools: We compare the *Accuracy* and effectiveness of our technique with other vulnerability detection approaches. We report in Table 6, the comparison of *TP* and false negative, while there was not a proper phenomenon to count *TP* and *FP* of these different types of approaches. Thus, we took these results from their own mentioned publications and a comparative study article.¹⁵ These selected approaches for comparison are, ReDeBug,¹⁰ Checkmarx³⁰, Deckard,²⁰ VulPecker,²¹ VulDeePecker,²² VCIPR,⁸ UDDY,¹⁵ and SQVDT (our approach). The reason we choose these approaches, because these are widely used.

ReDeBug takes some lines as a processing unit, and it uses a window set ($n = 4$) to slide it through the code by getting different hash functions for each window. Although ReDeBug can detect type-3 clones but which is not useful during vulnerability detection, thus, it has almost 17% of *FP* rate. Checkmarx uses data flow analysis, to recognize and detect vulnerabilities by human experts. This approach has a lot of human involvement for the static evaluation of vulnerabilities, so its quite far from perfect. DECKARD extracts all the main characteristic vectors from ASTs (for every file). This approach is slow and it uses extensive time subgraph isomorphism for vulnerability detection but this approach does not guarantee

³⁰<https://www.checkmarx.com/>

scalability. Furthermore, it has very low FP rate of 90%, which is not acceptable for vulnerability detection. VulPecker has a bit high FP rate of 15%. VulPecker uses diff's as an input for vulnerability detection, where diff is the difference files between a vulnerable code and its patched (changes) version. VulPecker is designed to detect vulnerabilities with diffs, it automatically detects whether a software contains a vulnerability or not. The main underlying of this approach is to leverage some features that characterize patches and code similarity algorithm. VulDeePecker has much lower FP rate (5%) than VulPecker, which substantially performs good for the pattern-based vulnerability detection approaches. Furthermore, note that the deep learning-based approaches mainly rely on the amount of the source code.

VCIPR is a very scalable vulnerability detection technique to detect vulnerabilities at function-level granularity. This technique extracts vulnerable functions from source code by tracing the patch files with respects to CVE-numbers. Then use these code fragments to detect vulnerabilities in other systems. VCIPR has also a low FP rate of 17%, and the author claims that it can be reduced up to 3% by combing all the functions detected against each CVE. VUDDY is another scalable vulnerability detection approach, which also detect vulnerabilities at function-level granularity. This technique has a high TP rate of 99% with no FP detection. This approach is twice faster than ReDeBug, which is quite effective. But this approach doesn't deal with the problem, in case of a vulnerability consists of multiple functions (not a single independent function) in a file. For example, sometime when a patch is released, this patch might consists of multiple changes in different source code files of a system as shown in Figure 2, there are three changes required in a file to overcome this vulnerability. Thus, multiple changes in different functions (not-independent) of a file formed a vulnerability. With respect to this statement sometimes to detect a single function (not-independent) cannot be claimed as a vulnerability has been detected. To overcome this problem, our approach (SQVDT) detects vulnerabilities at file level granularity with very low FP detection rate of 1%, which can be considered as 0% because from Table 2, we can see that for subject system *linux-5.1.4*, we evaluated all results and we got it 0%.

As our approach is based on Hadoop-environment, which consists of multiple nodes and can be extended very easily to hundreds of machines to handle a very large-scale code up to billions of projects. Up-to our knowledge, there is no technique which has been scaled, tested and detected vulnerabilities in almost 7-TB of source code. Thus, in overall comparison with other state-of-the-art-tools, our approach (SQVDT) performs better in efficiency as well as in scalability.

7 | RESULTS ANALYSIS: THREATS, VULNERABILITIES, AND EXPLOITS

In this section, we analyzed our results that are retrieved by our vulnerability detection technique. The exploit is the next step taken by an attacker after finding vulnerabilities. Thus, the threat is always there and it refers to the hypothetical event in which an attacker uses vulnerability. As we mentioned in the previous sections that we have discovered a lot of vulnerable files in thousands of OSS, in this section, we will just name and discuss some of them as a reference.

7.1 | Empirical evaluation

During vulnerability detection in OSS, we explored that a lot of software are having cloned code from other software, which are exposed as vulnerabilities. By taking an advantage of our scalable code clone detection technique, we investigate almost 152,823 wide range of projects. In this section, we will evaluate some of the real-world affected software by cloned code from old vulnerabilities. Moreover, we analyzed the root cause problem of expanding these vulnerabilities and draw an insight view that these vulnerabilities should be patched as early as possible. Some old vulnerabilities are always tend to remain unpatched in several software systems that can be a code clone. Mostly these vulnerabilities live for a long time until they are not detected. The cloned vulnerabilities can easily be triggered by some know exploits and they can cause a severe damage to the system. Patching these vulnerabilities is very challenging when multiple vendors are involved in this clone chain. Furthermore, it's very hard to manually patch every cloned vulnerability (function, file, component). So an automated system is required to detect and patch this vulnerable source code. According to result analysis, we classified these vulnerabilities into three categories.

- Intraproject clone vulnerabilities.
- Copied-Kernel clone vulnerabilities.
- Library-used clone vulnerabilities.

7.1.1 | Intraproject clone vulnerabilities

These vulnerabilities were most often appeared vulnerabilities in our results. As we detected vulnerabilities in thousands of open source projects of almost 7-TB (C/C++) of source code. These projects also includes different versions of one product. Most of the version were including Linux versions. Because our most of the vulnerability dataset belongs to Linux vulnerabilities, that's why we retrieved bundle of vulnerabilities from these version. Since, the vulnerability was published, we detected these vulnerabilities in almost all prevision versions of same product. For example, we detected vulnerabilities in two different versions of Linux (Linux-5.1.4 and Linux-5.3.7). Thus, it shows that vulnerable source code propagate from one system to another in the form of cloned code. Here is a list of detected CVEs in both versions (Linux-5.1.4 and Linux-5.3.7). The common vulnerabilities between these two operating systems are shown below. These results show that in the latest stable version of Linux-5.3.7, there are still some vulnerable files that are not patched yet.

Linux-5.1.4

CVE-2013-7421, CVE-2014-3184, CVE-2015-3212, CVE-2016-1583, CVE-2015-4178

Linux-5.3.7

CVE-2013-7421, CVE-2014-3184, CVE-2015-3212, CVE-2017-18218, CVE-2016-1583

7.1.2 | Copied-kernel clone vulnerabilities

Linux kernel is an oldest and largest OSS. Its distribution and different versions are spread to a number of open source projects and distributions, such as Android kernel, Ubuntu, Suse, Debian, IoT devices, mobile devices, and so forth. Some important characteristic of using Linux kernel is portability, features, process management, driver model, file system management, network stack, security, and so forth. Furthermore, this is becoming very adoptive ecology in IoT devices. We have observed that mostly it requires almost half a year to develop a new OS on the basis of Linux kernel. On the other hand, these Linux kernel-based devices are a bit behind the patching process because mostly when a vulnerability detected in kernel, a patch is released after a short time but the devices which are using these kernel versions are not informed about these updates or maybe the users ignore these patches. For example, we detected many kernel vulnerabilities in some smartphones software especially in OS which are kernel based. For example, vulnerable clone files detected in `Android_kernel_htc`³¹ and `FindX-8.1-kernel`³² shows that Linux kernel was cloned in their source code, which were patched in main Linux kernel but in smartphone's kernel these files still exist.

Android_kernel_htc

CVE-2013-0310, CVE-2013-7339, CVE-2014-8884, CVE-2013-4512, CVE-2012-3430, CVE-2012-2669, CVE-2016-10318, CVE-2012-4565, CVE-2013-7027, CVE-2013-1827, CVE-2014-5206, CVE-2014-5207, CVE-2013-0216, CVE-2009-3620, CVE-2012-2373, CVE-2012-6546, CVE-2015-2150, CVE-2013-1957, CVE-2012-5374, CVE-2012-5375, CVE-2013-7263, CVE-2013-7264, CVE-2013-7265, CVE-2013-7281, CVE-2010-3081, CVE-2014-7822, CVE-2014-9644, CVE-2013-7421

FindX-8.1-kernel

CVE-2018-6412, CVE-2017-16912, CVE-2014-3184, CVE-2013-7421, CVE-2017-16914, CVE-2015-3212, CVE-2017-18075, CVE-2016-1583, CVE-2015-4178, CVE-2017-15951, CVE-2018-18386, CVE-2017-17805, CVE-2017-18232, CVE-2016-9777, CVE-2017-9059, CVE-2017-11472, CVE-2018-17182

³¹https://github.com/CyanogenMod/android_kernel_htc_msm8974

³²<https://github.com/oppo-source/FindX-8.1-kernel-source>

7.1.3 | Library used clone vulnerabilities

Another kind of vulnerability detection was explored at library-level granularity, in which different software directly or indirectly use some open source libraries to speedup the development process. In practice, it takes place frequently, as these libraries (third-party) are developed to be reused. Any kind of software, can use these libraries. For example, only VLC-player used almost 90 third-party libraries, including LAME, FFmpeg, FLAC, and so forth. It's very common that many projects use some outdated libraries or the library files which are vulnerable or not patched yet. Some of the detected results have been uploaded on GitHub³³ as a reference. The name of these projects has been assigned a fake ID because of security and privacy reasons. Few discovered CVEs are displayed below as a reference.

Library Vulnerabilities

CVE-2016-8650, CVE-2016-7914, CVE-2017-12193, CVE-2015-8948, CVE-2010-2498

7.2 | Vulnerabilities verification

The announcement of patch means that there will be one less vulnerability in overall, thus will be an increase in the security. Typically patches reveal information about the security holes, therefore having an early access to a patch file may advantage to an attacker. There were two ways to verify the detected vulnerabilities, exploit generation, or the static source code analysis.

7.2.1 | Patch-based exploit generation

This is a practical automatic exploit generation based on a published patch file. In exploit generation, the same program P and P' where P' are the fixes of a vulnerability in P . The purpose is to generate exploit for the vulnerability fixed in P' . An automatic patch-based exploit is possible to generate. The exploit generation is mainly based on the observation of bugs, which are usually fixed by adding the missing checks. From the vulnerability source code and the patch files info, we know the exact fault in the source code. Thus, we can create a direct exploit by exploring that particular vulnerability, that is, DOS, execute code, overflow, sql injection, and so forth.

However, we know the detected vulnerabilities can be exploited, thus we can extend our way to generate particular kinds of attacks, by writing the specific conditions as a constraint. For example, by adding some assertions we predicate any solution that must overwrite the sensitive data structures. There are some shortcoming of automatic exploit generation for discovered vulnerabilities, that is, create our own exploit or find the already exploits list to create an exploit based on already discovered vulnerabilities, and so forth. So, we used static analysis to prove the discovered vulnerabilities.

7.2.2 | Patch-based static analysis

This is our adopted method to verify the discovered vulnerabilities. In this method, we explored the number of files on which patch files were applied against each detected vulnerability (CVE-number). For example, every vulnerability depends on the required changes in the source code. These changes varies from 1-LOC to N-LOC, 1-function (code fragment) to N-functions, and from 1-file to N-files. For example, there are two vulnerabilities detected in Linux-5.3.7 (CVE-2016-1583 and CVE-2014-3184). In CVE-2016-1583, there was only one file changed to overcome this vulnerability and in CVE-2014-3184 the changes applied on six files to overcome this vulnerability. Thus, to make sure that the discovered vulnerability is exact, we manually check the number of files on which changes were applied. For example, if the changes were applied on two files and we detected both similar files according to that particular vulnerability definition, its mean that our discovered vulnerability is right. And that system in which we detected that vulnerability is vulnerable and can be exploit by any attacker.

³³<https://github.com/znd15/Lib-vul-C12-sim-1>

7.3 | Implications of patch-based static analysis

In this section, we verify our detected and verified vulnerabilities in different software by using static analysis method.

7.3.1 | Vulnerabilities discovered in Linux

By exploring our results, we found CVE-2015-4178 an exact vulnerability in the a stable version of Linux-5.1.4. As we can see from the CVE-number this vulnerability is 4 years old. Even though its patch was released as shown in Figure 13, but this vulnerable file still exist in this version. There can be two reasons, the patch was not successfully applied or the code was cloned from some old versions of Linux. This vulnerability is in the Linux-kernel before 4.0.5. It does not ensure the consistency of a list data structure, which allows users to cause a “denial of service attack (system crash)” by leveraging user name space at root access for an umount-2 system call in fs/fs_pin.c. The file fs_pin, allows the possibility that m_list or the s_list go unused. This supports a lazily unmounting the locked mounts. Because entire unmounted subtree needs to stay until there is no user with references to a subtree. To guarantee that the fs_pin, m_list and s_list are initialized them in init_fs_pin to allow for make sure that the pin_insert_group does not touch them. Furthermore use hlist_del_init in the pin_remove, so that there is hlist_unhashed test to attempt an update the previous list item. Here is the list of detected files against each CVE including unpatched files.

```
diff --git a/fs/fs_pin.c b/fs/fs_pin.c
index b06c987..611b540 100644
--- a/fs/fs_pin.c
+++ b/fs/fs_pin.c
@@ -9,8 +9,8 @@ static DEFINE_SPINLOCK(pin_lock);
void pin_remove(struct fs_pin *pin)
{
    spin_lock(&pin_lock);
-   hlist_del(&pin->m_list);
-   hlist_del(&pin->s_list);
+   hlist_del_init(&pin->m_list);
+   hlist_del_init(&pin->s_list);
    spin_unlock(&pin_lock);
    spin_lock_irq(&pin->wait.lock);
    pin->done = 1;
```

FIGURE 13 CVE-2015-4178 patch file discovered in Linux-5.1.4 [Color figure can be viewed at wileyonlinelibrary.com]

Vulnerable files detected in Linux-5.1.4

- 95 files changed 1 file detected as not patched. (CVE-2013-7421)
 - originpath:/sourcecode/linux-5.1.4/arch/x86/crypto/twofish_glue.c
- 6 files changed, 4 files detected as not patched. (CVE-2014-3184)
 - originpath:/sourcecode/linux-5.1.4/drivers/hid/hid-cherry.c
 - originpath:/sourcecode/linux-5.1.4/drivers/hid/hid-monterey.c
 - originpath:/sourcecode/linux-5.1.4/drivers/hid/hid-petalynx.c
 - originpath:/sourcecode/linux-5.1.4/drivers/hid/hid-sunplus.c
- 3 files changed, 1 file detected as not patched. (CVE-2015-3212)
 - originpath:/sourcecode/linux-5.1.4/include/net/netns/sctp.h
- 1 file changed, 1 file detected as not patched. (CVE-2016-1583)
 - originpath:/sourcecode/linux-5.1.4/fs/ecryptfs/kthread.c
- 1 file changed, 1 file detected as not patched. (CVE-2015-4178)
 - originpath:/sourcecode/linux-5.1.4/fs/fs_pin.c

In these results, we can see that CVE-2015-4178 and CVE-2016-1583 only has one file, on which changes were applied to overcome the vulnerability. By using our approach we detected these individual files in Linux-5.1.4 and in latest version of Linux-5.3.7. Thus, we can consider these files as a vulnerable files in these versions.

Vulnerable files detected in Linux-5.3.7

95 files changed, 2 files detected as not patched. CVE-2013-7421

- originpath:/linux-5.3.7/arch/x86/crypto/aes_glue.c
- originpath:/linux-5.3.7/arch/x86/crypto/twofish_glue.c

6 files changed, 4 files detected as not patched. (CVE-2014-3184)

- originpath:/linux-5.3.7/drivers/hid/hid-cherry.c
- originpath:/linux-5.3.7/drivers/hid/hid-monterey.c
- originpath:/linux-5.3.7/drivers/hid/hid-petalynx.c
- originpath:/linux-5.3.7/drivers/hid/hid-sunplus.c

3 files changed, 1 file detected as not patched. (CVE-2015-3212)

- originpath:/linux-5.3.7/include/net/netns/sctp.h

2 files changed, 1 file detected as not patched. (CVE-2017-18218)

- originpath:/linux-5.3.7/drivers/net/ethernet/hisilicon/hns/hns_enet.h

1 file changed, 1 file detected as not patched. (CVE-2016-1583)

- originpath:/linux-5.3.7/fs/ecryptfs/kthread.c

7.3.2 | Vulnerabilities discovered in HTC-kernel and FindX-8.1-kernel

Similarly, we detected vulnerable files in these smartphone systems. We evaluated, and displayed some selected results as a reference. These results consist on single file against each CVE, which were patched in main Linux-kernel but remain unpatched in these smartphone systems. These files were analyzed statically that proves theses vulnerabilities as an exact vulnerabilities. Here is the list of these files with respect to their CVE-numbers.

Vulnerable files detected in Android-kernel-HTC

1 file changed, 1 file detected as not patched.

- originpath:/android-kernel-htc-14.1/net/ipv4/cipso_ipv4.c
clonepath:/CVE-2013-0310/root/net/ipv4/cipso_ipv4.c
- originpath:/android-kernel-htc-14.1/net/rds/ib.c
clonepath:/CVE-2013-7339/root/net/rds/ib.c
- originpath:/android-kernel-htc-14.1/drivers/media/dvb/ttusb-dec/ttusbdecfe.c
clonepath:/CVE-2014-8884/root/drivers/media/usb/ttusb-dec/ttusbdecfe.c
- originpath:/android-kernel-htc-14.1/arch/um/kernel/exitcode.c
clonepath:/CVE-2013-4512/root/arch/um/kernel/exitcode.c
- originpath:/android-kernel-htc-14.1/net/rds/recv.c
clonepath:/CVE-2012-3430/root/net/rds/recv.c
- originpath:/android-kernel-htc-14.1/tools/hv/hv_kvp_daemon.c
clonepath:/CVE-2012-2669/root/tools/hv/hv_kvp_daemon.c
- originpath:/android-kernel-htc-14.1/fs/crypto/policy.c
clonepath:/CVE-2016-10318/root/fs/crypto/policy.c
- originpath:/android-kernel-htc-14.1/net/ipv4/tcp_illinois.c
clonepath:/CVE-2012-4565/root/net/ipv4/tcp_illinois.c
- originpath:/android-kernel-htc-14.1/net/wireless/radiotap.c
clonepath:/CVE-2013-7027/root/net/wireless/radiotap.c
- originpath:/android-kernel-htc-14.1/net/dccp/ccid.h
clonepath:/CVE-2013-1827/root/net/dccp/ccid.h

Vulnerable files detected in FindX-8.1-kernel

1 file changed, 1 file detected as not patched.

- originpath:/FindX-8.1-kernel-master/drivers/video/fbdev/sbuslib.c
clonepath:/linux-kernel/CVE-2018-6412/root/drivers/video/fbdev/sbuslib.c
- originpath:/FindX-8.1-kernel-master/drivers/usb/usbip/stub_rx.c
clonepath:/linux-kernel/CVE-2017-16912/root/drivers/usb/usbip/stub_rx.c
- originpath:/FindX-8.1-kernel-master/drivers/usb/usbip/stub_tx.c
clonepath:/linux-kernel/CVE-2017-16914/root/drivers/usb/usbip/stub_tx.c
- originpath:/FindX-8.1-kernel-master/drivers/acpi/acpica/nsutils.c
clonepath:/linux-kernel/CVE-2017-11472/root/drivers/acpi/acpica/nsutils.c
- originpath:/FindX-8.1-kernel-master/include/linux/vmacache.h
clonepath:/linux-kernel/CVE-2018-17182/root/include/linux/vmacache.h
- originpath:/FindX-8.1-kernel-master/include/net/netns/sctp.h
clonepath:/linux-kernel/CVE-2015-3212/root/include/net/netns/sctp.h
- originpath:/FindX-8.1-kernel-master/crypto/pcrypt.c
clonepath:/linux-kernel/CVE-2017-18075/root/crypto/pcrypt.c
- originpath:/FindX-8.1-kernel-master/fs/ecryptfs/kthread.c
clonepath:/linux-kernel/CVE-2016-1583/root/fs/ecryptfs/kthread.c
- originpath:/FindX-8.1-kernel-master/fs/fs_pin.c
clonepath:/linux-kernel/CVE-2015-4178/root/fs/fs_pin.c
- originpath:/FindX-8.1-kernel-master/net/dns_resolver/dns_key.c
clonepath:/linux-kernel/CVE-2017-15951/root/net/dns_resolver/dns_key.c
- originpath:/FindX-8.1-kernel-master/drivers/tty/n_tty.c
clonepath:/linux-kernel/CVE-2018-18386/root/drivers/tty/n_tty.c

8 | THREATS TO VALIDITY

In this section, we discuss the main threats to the validity of our approach in SQVDT and how can we tackle these effects? During the detection of vulnerabilities, we considered each and every vulnerability equally important by not dividing them on the basis of its severity level, but still to check the severity level, we can track its CVE information individually. In the case of fingerprint indexing of vulnerability code dataset, our dataset does not cover all of the known vulnerabilities because some of the vulnerabilities are not in OSS, so the source code is not available. And our approach totally depends upon the vulnerability source code. Although the chosen vulnerabilities may cover more than 60% of all known open source vulnerabilities and almost 100% of Linux kernel but still this may considered as a threat to validity. The vulnerability dataset was collected from different sources against applied patch files, there can be a possibility that some CVE's referenced wrong patch files. But these cases are very rare because we get the referenced patch files from some authentic CVE websites.

9 | RELATED WORK

A lot of researcher have been working on the prediction of vulnerabilities by using modeling techniques, in which they collected the previously published CVE's and on the basis of this dataset they predict future occurring of vulnerability. Researchers in the field of software reliability have analyzed software defect rates. The growth model of software reliability relate the number of defects during testing time. In the most recent work, it has been focused on detecting vulnerabilities in OSS,^{5,14} meanwhile proposing the prediction models for future occurring vulnerabilities, which do not concern with source code but previously published CVE data.^{6,23-26} Detecting code clones in big code bases have been very useful to overcome security flaws.^{14,27-29} Prior research has also shown that there are many fake applications in android

markets.^{11,30-33} Some of these techniques have high false rate because they detect clones at different level of granularities and different types. Some of them are language depended. There are very few researchers, who study patch files and retrieve vulnerable source code then detect unpatched files from OSS. VFDETECT³⁴ proposed an approach based on innovative fingerprint model to detect vulnerabilities in the source code. ReDeBug¹⁰ takes a set of lines of code as its processing unit and slides a window of $n = 4$ through the source code and applies hash functions to each window. The clones detected by bloom filter, which save the hashes of every window. ReDeBug detects some of type-3 clones, which are the indication of more false rate. DECKARD²⁰ extracts all the characteristic vectors from ASTs (for every file). This approach is a bit slow and require extensive time subgraph isomorphism. Meanwhile DECKARD does not guarantee the scalability. Moreover, DECKARD has 90% FP rate, which is not acceptable for the detection of exact vulnerable files. VulPecker²¹ is an automatically checking system and it characterizes all the vulnerabilities with different set of features, after that it selects one code optimal similarity algorithms for vulnerability detection. It has detected almost 40 vulnerabilities, which were not registered in the NVD. But still its not good for massive vulnerability detection in OSS, because it is slow.

10 | CONCLUSION AND FUTURE WORK

In this article, we proposed a scalable vulnerability detection approach to detect vulnerabilities in different software at source code level. We extracted different features from the source code before creating an index of fingerprint, then we detected vulnerabilities in the subject system by applying feature matching filters on it. This approach is implemented practically and the experiments shows the effectiveness and scalability of our approach. In the first experiment, we took vulnerabilities as a subject system and detect vulnerabilities (almost 13,580 vulnerable files were retrieved) in 7-TB of source code (C/C++). In the second experiment, we detected vulnerabilities in few latest versions of OSS, that is, *Linux-5.1.4*, *Linux-5.3.7*, *Android-FindX-8.1-kernel*, and *Android_kernel_htc*. Some of the results are mentioned in this article with respect to CVE reference numbers. Our future concerns are to collect the vulnerability source code of all known OSS, extend this approach to support multiple languages. Furthermore, to predict and detect zero-day vulnerabilities, and so forth. The main focus of our approach was to achieve scalability, reliability, efficiency and to evaluate the source code on the basis of already published vulnerabilities. Furthermore, to perform source code security assessment to trace down the vulnerable code, to find out “how these vulnerabilities transfer from one system to another?” and “how these vulnerabilities remain unfixed for long time?”

ORCID

Junaid Akram  <https://orcid.org/0000-0003-1752-6124>

REFERENCES

1. Avery J, Wallrabenstein JR. Formally modeling deceptive patches using a game-based approach. *Comput Secur*. 2018;75:182-190.
2. Brumley D, Poosankam P, Song D, Zheng J. Automatic patch-based exploit generation is possible: techniques and implications. Paper presented at: Proceedings of the 2008 IEEE Symposium on Security and Privacy (sp 2008). Oakland, California: IEEE; 2008:143-157.
3. Akram J, Mumtaz M, Luo P. IBFET: index-based features extraction technique for scalable code clone detection at file level granularity. *Softw Pract Exper*. 2020;50(1):22-46.
4. Akram J, Ping L. How to build a vulnerability benchmark to overcome cyber security attacks. *IET Inf Secur*. 2019;14(1):60-71.
5. Alhazmi OH, Malaiya YK, Ray I. Measuring, analyzing and predicting security vulnerabilities in software systems. *Comput Secur*. 2007;26(3):219-228.
6. Gorbenko A, Romanovsky A, Tarasyuk O, Biloborodov O. Experience report: study of vulnerabilities of enterprise operating systems. Paper presented at: Proceedings of the 2017 IEEE 28th International Symposium on Software Reliability Engineering (ISSRE). Toulouse, France: IEEE; 2017:205-215.
7. Mumtaz M, Akram J, Ping L. An RSA based authentication system for smart IoT environment. Paper presented at: Proceedings of the 2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 17th International Conference on Smart City; IEEE 5th International Conference on Data Science and Systems (HPCC/SmartCity/DSS). ZhangJiaJie, China: IEEE; 2019:758-765.
8. Akram J, Liang Q, Ping L. VCIPR: vulnerable code is identifiable when a patch is released (Hacker's Perspective). Paper presented at: Proceedings of the 2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST). Xian, China: IEEE; 2019:402-413.
9. Guo H, Wang YY, Pan ZL, Liu SW. Research on detecting windows vulnerabilities based on security patch comparison. Paper presented at: Proceedings of the 2016 Sixth International Conference on Instrumentation & Measurement, Computer, Communication and Control (IMCCC). Harbin, Heilongjiang, China: IEEE; 2016:366-369.

10. Jang J, Agrawal A, Brumley D. ReDeBug: finding unpatched code clones in entire os distributions. Paper presented at: Proceedings of the 2012 IEEE Symposium on Security and Privacy. San Francisco Bay Area, California: IEEE; 2012:48-62.
11. Akram J, Shi Z, Mumtaz M, Luo P. DroidCC: a scalable clone detection approach for android applications to detect similarity at source code level. Paper presented at: Proceedings of the 2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC). Tokyo, Japan: IEEE; 2018:100-105.
12. Ng YY, Zhou H, Ji Z, Luo H, Dong Y. Which android app store can be trusted in China? Paper presented at: Proceedings of the 2014 IEEE 38th Annual Computer Software and Applications Conference. Vasteras, Sweden: IEEE; 2014:509-518.
13. Juergens E, Deissenboeck F, Hummel B, Wagner S. Do code clones matter? Paper presented at: Proceedings of the 2009 IEEE 31st International Conference on Software Engineering. Vancouver, BC: IEEE; 2009:485-495.
14. Sheneamer A, Kalita J. A survey of software clone detection techniques. *Int J Comput Appl*. 2016;137 (10):0975-8887.
15. Kim S, Woo S, Lee H, Oh H. VUDDY: a scalable approach for vulnerable code clone discovery. Paper presented at: Proceedings of the 2017 IEEE Symposium on Security and Privacy. San Jose, California: IEEE; 2017:595-614.
16. Alqahtani SS, Eghan EE, Rilling J. Tracing known security vulnerabilities in software repositories—a semantic web enabled modeling approach. *Sci Comput Prog*. 2016;121:153-175.
17. Johnston R, Sarkani S, Mazzuchi T, Holzer T, Eveleigh T. Bayesian-model averaging using MCMCBayes for web-browser vulnerability discovery. *Reliab Eng Syst Saf*. 2019;183:341-359.
18. Sharma R, Singh RK. Vulnerability discovery in open-and closed-source software: a new paradigm. *Software Engineering*. Singapore: Springer; 2019:533-539.
19. Jogi VD, Sinha A. Performance evaluation of MySQL, Cassandra and HBase for heavy write operation. Paper presented at: Proceedings of the 2016 3rd International Conference on Recent Advances in Information Technology (RAIT). Dhanbad, India: IEEE; 2016:586-590.
20. Jiang L, Mishserghi G, Su Z, Glondu S. Deckard: scalable and accurate tree-based detection of code clones. Paper presented at: Proceedings of the 29th International Conference on Software Engineering, IEEE Computer Society. Shanghai China: IEEE; 2007:96-105.
21. Li Z, Zou D, Xu S, Jin H, Qi H, Hu J. VulPecker: an automated vulnerability detection system based on code similarity analysis. Paper presented at: Proceedings of the 32nd Annual Conference on Computer Security Applications. Los Angeles California USA: ACM; 2016:201-213.
22. Li Z, Zou D, Xu S, et al. VulDeePecker: a deep learning-based system for vulnerability detection; 2018. arXiv preprint arXiv:1801.01681. <https://doi.org/10.14722/ndss.2018.23158>.
23. Jabeen G, Ping L, Akram J, Shah AA. An integrated software vulnerability discovery model based on artificial neural network. *SEKE*. Lisbon, Portugal: International Conference on Software Engineering and Knowledge Engineering; 2019:349-458.
24. Kim D, Tao Y, Kim S, Zeller A. Where should we fix this bug? a two-phase recommendation model. *IEEE Trans Softw Eng*. 2013;39(11):1597-1610.
25. Nappa A, Johnson R, Bilge L, Caballero J, Dumitras T. The attack of the clones: a study of the impact of shared code on vulnerability patching. Paper presented at: Proceedings of the 2015 IEEE Symposium on Security and Privacy; 2015:692-708; IEEE.
26. Kim S, Lee H. Software systems at risk: an empirical study of cloned vulnerabilities in practice. *Computers and Security*. Elsevier; 2018;77:720-736.
27. Juergens E, Deissenboeck F, Hummel B. CloneDetective-A workbench for clone detection research. Paper presented at: Proceedings of the 2009 IEEE 31st International Conference on Software Engineering Computer Society. Washington, DC: IEEE; 2009:603-606.
28. Sajnani H, Saini V, Lopes C. A parallel and efficient approach to large scale clone detection. *J Softw Evolut Process*. 2015;27(6):402-429.
29. Akram J, Shi Z, Mumtaz M, Luo P. DCCD: an efficient and scalable distributed code clone detection technique for big code. *SEKE*. California, USA: KSI Research Inc; 2018:354-359.
30. Akram J, Mumtaz M, Jabeen G, Luo P. DroidMD: an efficient and scalable android malware detection approach at source code level. *Int J Inf Comput Secur*. 2019;11(1). <https://doi.org/10.1504/IJICS.2019.10020453>.
31. Chen K, Liu P, Zhang Y. Achieving accuracy and scalability simultaneously in detecting application clones on android markets. Paper presented at: Proceedings of the 36th International Conference on Software Engineering. Hyderabad India: ACM; 2014:175-186.
32. Zhou Y, Jiang X. Dissecting android malware: characterization and evolution. Paper presented at: Proceedings of the 2012 IEEE Symposium on Security and Privacy; 2012:95-109; IEEE.
33. Zhou W, Zhou Y, Jiang X, Ning P. Detecting repackaged smartphone applications in third-party android marketplaces. Paper presented at: Proceedings of the 2nd ACM Conference on Data and Application Security and Privacy; 2012:317-326; ACM.
34. Liu Z, Wei Q, Cao Y. VFDETECT: a vulnerable code clone detection system based on vulnerability fingerprint. Paper presented at: Proceedings of the 2017 IEEE 3rd Information Technology and Mechatronics Engineering Conference. Chongqing China: IEEE; 2017:548-553.

How to cite this article: Akram J, Luo P. SQVDT: A scalable quantitative vulnerability detection technique for source code security assessment. *Softw Pract Exper*. 2020;1–25. <https://doi.org/10.1002/spe.2905>